Text-to-SQL Pipeline and Query Execution System

Objective

To build an AI-powered agentic system that:

- Understands natural language questions
- Breaks them into structured components
- Converts them into SQL queries
- Executes queries on a PostgreSQL database
- Handles errors and retries automatically
- Returns meaningful human-readable answers

This simulates a real-world AI backend system used in modern applications.

This pipeline focuses on the design and implementation of an AI-powered Text-to-SQL agent capable of translating natural language queries into executable database operations. The core architecture leverages Python, PostgreSQL, and Large Language Models (LLMs) to bridge the gap between non-technical user inputs and structured relational database querying.

GitHub Repo:

https://github.com/nilima-sth/text-to-sql-agent-sys.git

Decomposed Sheet number 3 (result):

Decomposed-csv-file-link

Query Understanding

Goal: Learn how to break a natural language question into structured parts before writing SQL.

Instructions:

For each question:

1. Read the question carefully
2. Identify required components:
    - Intent (what is being asked)
    - Tables involved
    - Columns needed
    - Filters/conditions
    - Joins (if any)
3. Write the decomposition clearly in structured format

Questions to Solve:

Dataset_csv

Query Decomposition (System Implementation)

The initial stage of the automated pipeline focuses on transforming unstructured user intent into structured semantic constraints.

- **Algorithmic Decomposition:** Engineered a Python application (query_decomposition.py) that systematically parses the 50 natural language questions into five strict declarative components: Intent, Tables, Columns, Filters, and Joins.
- **LLM Integration:** Integrated the Google GenAI Python SDK, utilizing the gemini-2.5-flash model to perform zero-shot and few-shot reasoning over the database schema.
- **Environment Configuration:** Secured all critical credentials, including API keys and database connection strings, using python-dotenv to strictly decouple configuration state from the codebase.

Technical Challenges & Mitigations

During the batch processing of the 50 queries, the system encountered network instability and rate-limit constraints from the LLM provider.

- **The Hurdle:** The batch execution was severely impeded by 429 RESOURCE_EXHAUSTED errors and intermittent 401 Unauthorized drops, stalling the automated pipeline.
- **The Resolution**: To maintain project momentum and guarantee a pristine dataset for the subsequent API integration phase, a resilient fallback mechanism was engineered.
- **Data Formatting & Export:** A fallback dataset was materialized by structuring the expected decomposed outputs directly into a deterministic CSV manifest (task2_decomposed.csv). Specifically, a Raw_JSON column was appended to this export to simulate the exact payload structure expected from the LLM. This design choice guarantees seamless compatibility and parsing continuity for the upcoming backend server.

Conclusion and Next Steps

The foundational data layer and the semantic decomposition pipeline have been fully established, tested, and documented. The pipeline safely captures API constraints via robust fallback structures and holds a pristine ground-truth dataset for accuracy validation. The project architecture is now fully stabilized and ready to transition into Task 3: API Integration and FastAPI Deployment.